

Relatório técnico - Jogo (terceira avaliação)

Bianca da Silva Magalhães Pinheiro¹, Mayra Gabriela Sousa da Silva²

Departamento de Ciência da Computação
Universidade Federal do Piauí (UFPI) – Teresina, PI – Brasil

(20249012612)¹ - (20249037694)²

Resumo. *Este trabalho apresenta o desenvolvimento de um jogo 3D do gênero FPS (First-Person Shooter) utilizando C++ e OpenGL. O jogo inclui movimentação em primeira pessoa, sistema de mira, disparos, detecção de colisões e inimigos com comportamento de perseguição, além de um Boss como desafio final. O projeto aplica conceitos de Computação Gráfica, modelagem 3D, programação orientada a objetos e interação em tempo real, demonstrando o uso da OpenGL no desenvolvimento de jogos digitais.*

1. Introdução

O projeto consiste no desenvolvimento de um jogo 3D do gênero FPS (First-Person Shooter), ambientado em um cenário de ficção científica, no qual o jogador enfrenta Troopers e, ao final, um Boss. O jogo possui movimentação em primeira pessoa, sistema de mira, disparos e inteligência artificial básica para os inimigos. A partida termina com a derrota do Boss ou quando a vida do jogador chega a zero.

O trabalho teve como objetivo aplicar conceitos de Computação Gráfica e Programação Orientada a Objetos no desenvolvimento de um jogo funcional em C++ e OpenGL, integrando câmera em primeira pessoa, combate, renderização 3D, colisões e interface gráfica.

2. Materiais

O jogo foi desenvolvido utilizando a linguagem C++, que oferece alto desempenho e amplo suporte para aplicações gráficas e interativas em tempo real. A principal biblioteca empregada foi a OpenGL, responsável pela renderização dos modelos tridimensionais, aplicação de transformações geométricas, iluminação da cena e exibição dos elementos gráficos do jogo.

Para o gerenciamento da janela da aplicação e da interação com o usuário, foi utilizada a biblioteca GLUT (OpenGL Utility Toolkit). Essa biblioteca possibilita a captura de eventos de teclado e mouse, o controle da câmera em primeira pessoa e a atualização contínua da cena por meio do loop principal do jogo.

O projeto também faz uso das bibliotecas TinyObjLoader e stb_image, empregadas para o carregamento de modelos 3D no formato OBJ e de texturas aplicadas aos objetos da cena. Essas bibliotecas permitem a criação de ambientes mais detalhados e visualmente atrativos.

Além das bibliotecas externas, foram desenvolvidas classes específicas para organizar os componentes do jogo, incluindo câmera, cenário, inimigos, sistema de disparos, colisões, menu inicial e tela final. Essa estrutura modular facilitou a implementação das mecânicas principais, como movimentação em primeira pessoa, perseguição de inimigos, sistema de combate, controle de vida do jogador e condição de vitória por meio da derrota do Boss final.

3. Métodos

Métodos e Funcionalidades Implementadas

O desenvolvimento do jogo foi baseado na integração de diferentes técnicas de Computação Gráfica e programação de jogos, permitindo a criação de um ambiente tridimensional interativo e funcional.

Movimentação do Jogador

A movimentação é processada no callback `glutKeyboardFunc`, implementado em `Camera.cpp` na função `movimentacaoTeclado()`. O vetor de direção da câmera é calculado a partir dos ângulos `anguloY` (yaw horizontal) e `anguloX` (pitch vertical):

`dirX = sin(anguloY_rad) * cos(anguloX_rad)`

`dirZ = -cos(anguloY_rad) * cos(anguloX_rad)`

As teclas W/S movem o jogador ao longo desse vetor; A/D deslocam lateralmente pelo vetor perpendicular. A confirmação do movimento só ocorre se a nova posição passar por duas validações: `podeMoverPara()` (não atravessa paredes) e `podeMoverContraInimigos()` (não atravessa inimigos vivos).

Controle de Câmera

A câmera é do tipo primeira pessoa (FPS). O botão direito do mouse, mantido pressionado, ativa o modo de rotação. O callback `glutMotionFunc` chama `movimentacaoMouse()`, que calcula os deltas de posição do cursor e os traduz em variações angulares:

`anguloY += deltaX * sensibilidade;`

`anguloX -= deltaY * sensibilidade; // clampeado em [-45°, +45°]`

A função `visao()` constrói o vetor de direção 3D completo e o passa a `gluLookAt()`, atualizando a matriz de visão a cada frame.

Sistema de Colisões

Três tipos de colisão foram implementados em `Colisao.cpp`:

Colisão com paredes (`podeMoverPara`): o mapa é representado por retângulos AABB (Axis-Aligned Bounding Boxes) definidos estaticamente. Antes de cada movimento, verifica-se se a nova posição (x, z) está contida em alguma área permitida — corredor inicial, sala do meio, corredor final e sala do boss.

Colisão jogador-inimigo (`podeMoverContraInimigos`): distância mínima de 2.0 unidades entre o jogador e qualquer inimigo vivo, impedindo sobreposição.

Colisão bala-inimigo (`atingiuInimigoSegmento`): usa teste de distância ponto-segmento. O trecho percorrido pela bala em cada frame é tratado como um segmento de reta, e calcula-se a distância mínima entre esse segmento e o centro do inimigo. Isso evita o problema de tunneling (bala rápida passando por cima do alvo):

`t = clamp(dot(P - A, B - A) / |B - A|², 0, 1)`

`Pprox = A + t * (B - A)`

acerto se `|Pprox - centro_inimigo| ≤ raio`

Inteligência Artificial dos Inimigos

A IA dos stormtroopers é implementada em `Inimigo.cpp`, no método `atualizar()`, chamado a cada frame. O comportamento segue três etapas:

Orientação: `atan2(dx, dz)` calcula o ângulo `rotY` para que o inimigo olhe sempre na direção do jogador.

Perseguição: se a distância ao jogador for ≥ 5.0 unidades, o inimigo avança com velocidade de 0.02 unidades/frame. O movimento é bloqueado se colidissem com outro inimigo (separação de grupo com raio de 1.6 unidades) ou se saísse das áreas permitidas.

Ataque: `atacarJogador()` verifica se o inimigo está a menos de 6.5 unidades do jogador. Se sim, e se o intervalo desde o último disparo superar 1000 ms (medido com `std::chrono`), chama

atirar.dispararInimigo(), criando um projétil que voa em direção ao jogador no plano XZ. O boss tem comportamento mais simples: causa 5 pontos de dano por segundo ao jogador enquanto estiver vivo, gerenciado por um timer em main.cpp.

Sistema de Disparo

A classe Atirar gerencia um `std::vector` de structs Tiro. Cada tiro armazena posição atual, posição anterior (para o teste de segmento), direção 3D normalizada, distância percorrida e flags de estado (ativo, tiroInimigo).

O método disparar() calcula a direção 3D a partir dos dois ângulos da câmera, garantindo que o projétil siga exatamente o centro da mira:

```
dirX = sin(anguloY_rad) * cos(anguloX_rad)
```

```
dirY = sin(anguloX_rad)
```

```
dirZ = -cos(anguloY_rad) * cos(anguloX_rad)
```

O método atualizar() avança todos os tiros e, para os do jogador, invoca atingiuInimigoSegmento(). Ao detectar colisão, o tiro é imediatamente desativado (`ativo = false`) e o inimigo recebe 20 pontos de dano. Se o dano for letal, o contador de kills é incrementado. Tiros que ultrapassam 7.0 unidades sem acertar nenhum alvo são descartados.

Carregamento e Renderização de Modelos 3D

O projeto usa dois carregadores distintos:

Modelo.cpp — arquivos OBJ: usa `tiny_obj_loader` para parsear vértices, normais, coordenadas de textura e materiais .MTL. A geometria é compilada em uma Display List OpenGL (`glNewList / glCallList`) na primeira chamada, evitando reenvio de dados à GPU a cada frame. A coordenada V é invertida ($1.0 - v$) para corrigir a diferença de convenção entre OpenGL e o formato OBJ.

ModeloAnimado.cpp — arquivos GLTF: usa Assimp com as flags `aiProcess_Triangulate`, `aiProcess_FlipUVs`, `aiProcess_GenNormals` e `aiProcess_JoinIdenticalVertices`. O grafo de cena é percorrido recursivamente (`desenharNo` → `desenharMesh`), desenhando cada mesh com sua textura difusa ou cor de fallback cinza.

Iluminação e Sombreamento

A iluminação é configurada em `init()` usando o modelo de Phong do pipeline fixo do OpenGL:

```
cpp
```

```
glEnable(GL_LIGHTING);
```

```
glEnable(GL_LIGHT0);
```

`GL_LIGHT0` é uma fonte posicional ($w = 1.0$) em (0, 10, 10), com componente ambiente de (0.7, 0.7, 0.7) — que garante que nenhuma área do corredor fique completamente escura — e componente difusa de (1.0, 1.0, 1.0), realçando os volumes dos stormtroopers e da arquitetura.

Para materiais sem textura, as propriedades `GL_AMBIENT`, `GL_DIFFUSE`, `GL_SPECULAR` e `GL_SHININESS` são lidas do arquivo .MTL e aplicadas via `glMaterialfv()`. O sombreamento é calculado automaticamente com base nas normais por vértice. Para superfícies texturizadas, `glColor3f(1,1,1)` garante que a textura não seja modulada pela cor do material. Em todas as partes 2D (HUD, mira, menus), a iluminação é desabilitada com `glDisable(GL_LIGHTING)`.

Texturas

O pipeline de texturização segue as etapas:

Decodificação: `stbi_load()` carrega PNG ou JPEG em buffer de bytes, detectando automaticamente canais (3 = RGB, 4 = RGBA).

Upload para GPU: `glGenTextures / glBindTexture / glTexImage2D` envia os dados para um objeto de textura OpenGL.

Filtragem: GL_LINEAR para minimização e magnificação, resultando em interpolação bilinear suave sem serrilhado.

Mapeamento: coordenadas (u, v) enviadas por glTexCoord2f() antes de cada vértice dentro dos blocos GL_TRIANGLES.

O projeto carrega texturas PBR do cenário (baseColor, metallicRoughness, normal maps), porém no pipeline fixo do OpenGL apenas o mapa de cor difusa é efetivamente aplicado — os mapas de roughness e normal estão presentes nos assets mas exigiriam shaders GLSL para serem processados.

ambiente tridimensional e evita problemas de sobreposição indevida entre modelos.

Mira e HUD

A mira (mira.cpp) é desenhada em modo 2D com gluOrtho2D() após desabilitar o depth test. É composta por: círculo externo (GL_LINE_LOOP), quatro riscos externos (GL_LINES), quatro marcadores de canto internos e uma bolinha central sólida (GL_TRIANGLE_FAN). A cor muda de branco para vermelho quando um inimigo está na linha de tiro — detecção feita em inimigoNaDirecaoCamera() via produto escalar entre o vetor de direção da câmera e o vetor câmera-inimigo.

Abaixo da mira exibe-se uma mini barra de vida do inimigo focado, com gradiente de cor verde → amarelo → vermelho proporcional à vida restante. O HUD de kills usa duas camadas de texto sobrepostas (sombra preta deslocada + texto branco) para legibilidade sem fundo opaco.

Sistema de Spawn e Progressão

O mapa é dividido em cinco zonas. A função verificarSpawnsPorMapa() é chamada a cada tick (16 ms) e avalia a posição X da câmera para disparar grupos de inimigos progressivamente:

Condição	Zona	Quantidade
Início do jogo	CorredorInicial1	3 inimigos
camX ≤ 5.0	CorredorInicial2	4 inimigos
camX ≤ -2.0	SalaMeio	4 inimigos
camX ≤ -10.0	CorredorFinal	3 inimigos
camX ≤ -26.0 e todos mortos	Boss (Darth Vader)	1 boss

Cada inimigo é posicionado em coordenada aleatória dentro da zona, com validação de sobreposição (distância mínima 2.2 unidades) e verificação de área permitida.

Modelos 3D e Texturas

O cenário, os inimigos e demais elementos gráficos são representados por modelos tridimensionais carregados a partir de arquivos no formato OBJ. As texturas associadas aos modelos são carregadas e aplicadas durante a renderização, aumentando o nível de detalhamento visual e tornando o ambiente mais próximo de um cenário real.

Efeitos Visuais de Interface

Quando a vida do jogador cai para 25 pontos ou menos, é ativado um efeito de vinheta vermelha nas bordas (desenharEfeitoSangue). A intensidade é proporcional ao dano ($t = 1 - \text{vida}/25$) e a transparência decai quadraticamente da borda para o centro, criando uma borda densa e um miolo transparente. O efeito é desenhado com 18 faixas GL_QUADS em cada uma das quatro

bordas.

As telas de vitória e derrota (TelaFinal.cpp) implementam efeitos adicionais: raios giratórios com GL_TRIANGLES e alpha pulsante senoidal, partículas de confete animadas, estrelas de conquista desenhadas com GL_TRIANGLE_FAN de 10 vértices, scanlines CRT animadas e títulos com múltiplas camadas de glow (deslocamentos consecutivos de posição com alpha decrescente). Toda a animação é baseada em tempo contínuo via glutGet(GLUT_ELAPSED_TIME).

4. Resultados e Discussão

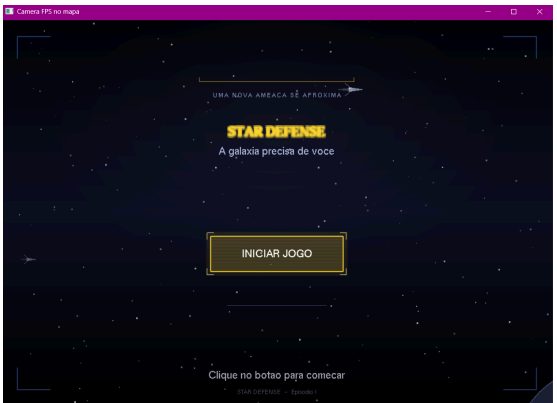


Figura 1. Tela de início



Figura 2. Tela de menu



Figura 3. Tela principal do jogo



Figura 4. Tela do inimigo final

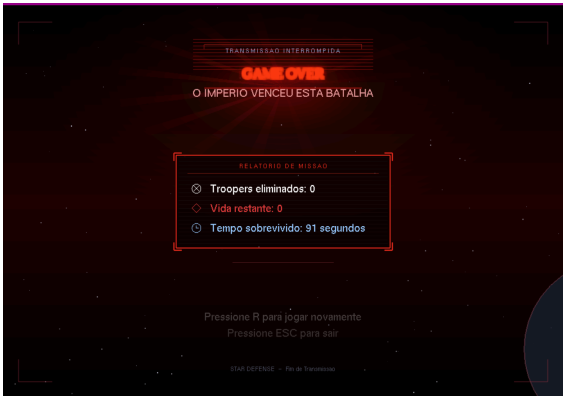


Figura 5. Tela de game over



Figura 2. Tela de missão concluída

5. Conclusão

Durante o desenvolvimento do jogo, alguns desafios exigiram ajustes e testes contínuos. A implementação das colisões foi uma das principais dificuldades, pois foi necessário impedir que o jogador e os inimigos atravessassem paredes e outros objetos do cenário, além de garantir uma movimentação natural. Outro desafio foi o carregamento e a aplicação das texturas nos modelos 3D, envolvendo a organização dos arquivos e a configuração correta dos materiais. Também foram realizados diversos ajustes na movimentação da câmera, dos projéteis e da inteligência artificial dos inimigos, buscando tornar o deslocamento e as interações mais fluidos e compatíveis com a proposta de um jogo FPS.

6. Referências

- Assimp Documentation. Official Documentation. Disponível em: <https://the-asset-importer-lib-documentation.readthedocs.io/>. Acesso em: 20 jun. 2026.
- Khronos Group. OpenGL Documentation. Disponível em: <https://www.khronos.org/opengl/>. Acesso em: 19 jun. 2026.
- LearnOpenGL. LearnOpenGL. Disponível em: <https://learnopengl.com/>. Acesso em: 20 jun. 2026.
- SYOYO. Tinyobjloader. GitHub, 2026. Disponível em: <https://github.com/tinyobjloader/tinyobjloader>. Acesso em: 21 jun. 2026.